PROJET V2V

Simulateur de communication Vehicle-to-Vehicle

Visualisation temps réel d'un trafic urbain et d'un réseau de communication inter-véhicules

Rapport	Documentation technique détaillée du projet
Technologie	C++20 · OpenGL 3.3 Core · CMake
Volume de code	8 318 lignes C++ · 64 fichiers source
Date	Mai 2026

Sommaire

1. Introduction et objectifs du projet
2. Pile technologique et système de build
3. Architecture générale du logiciel
4. Module graphique — fenêtre, rendu, caméra
5. Module cartographique — tuiles OpenStreetMap
6. Graphe routier — import OSM et visualisation
7. Simulation du trafic — véhicules et A*
8. Systèmes de trafic — feux, accidents, densité
9. Réseau V2V — liens radio et diffusions
10. Optimisation spatiale et performance
11. Interface utilisateur ImGui
12. Météo et effets d'ambiance
13. Méthode de développement par étapes
14. Statistiques du projet
15. Vérification, état du build et pistes d'évolution

1. Introduction et objectifs du projet

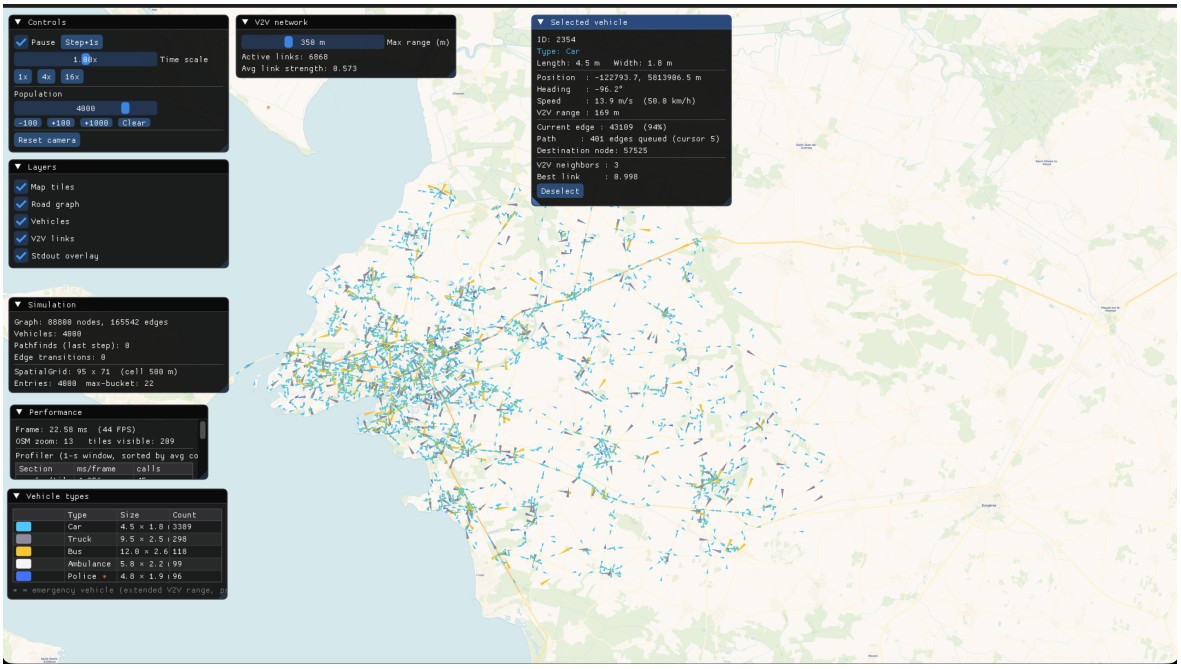
Le projet **V2V** est un simulateur graphique temps réel qui modélise la circulation de véhicules sur un réseau routier réel et la communication *Vehicle-to-Vehicle* qui s'établit dynamiquement entre eux. L'application charge un extrait de carte OpenStreetMap, en extrait le graphe des rues, y fait circuler plusieurs milliers de véhicules autonomes, et affiche en surimpression le réseau de liaisons radio qui relie les véhicules suffisamment proches les uns des autres.

L'objectif technique central était double : produire une **visualisation fluide** (cible de 60 images par seconde) tout en simulant un nombre élevé d'agents (cible de **2000 véhicules et plus**), et bâtir une architecture **modulaire et extensible** où chaque sous-système (carte, graphe, simulation, réseau, interface) reste indépendant et testable.

Au-delà du noyau de simulation, le projet a progressivement intégré des systèmes de trafic réalistes — feux de signalisation cycliques, génération d'accidents, carte de densité — ainsi que des effets d'ambiance (pluie, brouillard) qui influencent à la fois le rendu et le comportement physique des véhicules et des ondes radio.

> **En une phrase**
>
> V2V transforme une carte OpenStreetMap statique en un laboratoire vivant où l'on observe, contrôle et mesure un trafic dense et son réseau de communication, image par image, à la vitesse de l'œil.

Aperçu de l'application



Capture de l'application : graphe routier, véhicules et liens V2V rendus en temps réel.

2. Pile technologique et système de build

Le simulateur est écrit en **C++20** et s'appuie sur **OpenGL 3.3 Core Profile** pour le rendu accéléré par le processeur graphique. Le langage moderne a été retenu pour ses apports concrets : `constexpr` étendu pour les tables de constantes, structures de données compactes, et la macro `__VA_OPT__` utilisée par le système de journalisation. Toutes les dépendances externes sont récupérées automatiquement par CMake via **FetchContent** : aucune installation manuelle n'est requise, hormis Python 3 (générateur du loader OpenGL) et libcurl (présent par défaut sur macOS).

Bibliothèques utilisées

Bibliothèque	Version	Rôle dans le projet
GLFW	3.4	Création de la fenêtre, contexte OpenGL, gestion clavier/souris
GLAD 2	2.0.6	Chargement des pointeurs de fonctions OpenGL 3.3 Core
GLM	1.0.1	Mathématiques : vecteurs, matrices, transformations
Dear ImGui	1.91.5	Interface utilisateur immédiate (panneaux de contrôle)
stb_image	master	Décodage des images PNG (tuiles cartographiques)
pugixml	1.14	Analyse rapide des fichiers XML OpenStreetMap
libcurl	système	Téléchargement HTTP des tuiles raster OSM

Chaîne de compilation

Le projet se configure et se compile en deux commandes. Le premier `configure` clone les dépendances (quelques minutes) ; les compilations suivantes sont incrémentales.

```
cmake -B build -S .
cmake --build build -j
./build/bin/V2V data/mulhouse.osm 2000
```

CMake produit un exécutable unique, V2V, compilé en mode **Release** par défaut avec les avertissements stricts activés (`-Wall -Wextra -Wpedantic`). Les fichiers de shaders GLSL sont copiés à côté du binaire après chaque build, et le chemin du dossier source est aussi exposé comme définition de compilation pour permettre l'exécution directe depuis l'arborescence de développement.

3. Architecture générale du logiciel

Le code est organisé en **sept modules** aux responsabilités nettement séparées, regroupés sous l'espace de noms `v2v`. Le principe directeur est la séparation entre **l'état** (les données de simulation, sans aucune dépendance graphique) et le **rendu** (qui lit cet état pour produire des images). Ainsi le moteur de simulation peut tourner, être testé ou accéléré indépendamment de l'affichage.

Module	Responsabilité	Lignes
graphics	Fenêtre, contexte OpenGL, renderer, caméra 2D, shaders	857
map	Projection Mercator, cache et chargement des tuiles OSM	926
graph	Import OpenStreetMap, graphe routier, rendu des routes	713
simulation	Véhicules, A*, moteur de pas, feux, accidents, densité	1 618
v2v	Réseau de liens radio, modèle de signal, diffusions	718
ui	Panneaux ImGui de contrôle, de mesure et d'inspection	1 732
utils	Grille spatiale, profileur, météo, pluie, journalisation	848

La boucle principale

Le cœur de l'application est une **boucle à delta-temps** : à chaque tour, l'écart de temps réel écoulé depuis l'image précédente est mesuré, puis utilisé pour faire avancer la simulation d'une quantité physiquement cohérente, indépendamment de la fréquence d'affichage. Chaque image enchaîne, dans l'ordre : traitement des entrées clavier/souris, avancement de la simulation, reconstruction de la grille spatiale, calcul du réseau V2V, puis l'empilement des couches de rendu (tuiles, carte de densité, graphe, véhicules, liens, diffusions, pluie, brouillard) et enfin l'interface ImGui.

Un facteur d'échelle temporelle (*time scale*) permet de ralentir ou d'accélérer la simulation de 0,0625× à 16× sans toucher au taux de rafraîchissement, et la simulation peut être mise en pause tout en conservant un affichage et une caméra pleinement interactifs.

4. Module graphique — fenêtre, rendu, caméra

Window — fenêtre et contexte

La classe `Window` est une enveloppe **RAII** autour d'une fenêtre GLFW et de son contexte OpenGL. Elle initialise GLFW à la création de la première fenêtre et le libère à la destruction de la dernière. Les entrées (clavier, souris, molette, redimensionnement) sont transmises au reste du programme par des fonctions de rappel (`std::function`), ce qui découple complètement la boucle principale de l'API C de GLFW.

Renderer — rendu 2D de base

Le `Renderer` détient l'état OpenGL minimal partagé : le loader GLAD, un VAO/VBO de quadrilatère unitaire, et un shader élémentaire (transformation plus couleur uniforme). Tous les renderers spécialisés (tuiles, graphe, véhicules, liens) s'appuient sur le même contexte. Le rendu suit un protocole clair : `beginFrame` téléverse la matrice de caméra, puis les primitives sont dessinées, puis `endFrame`.

Camera2D — caméra orthographique

La `Camera2D` est une caméra orthographique 2D avec déplacement (*pan*) et zoom. Le **zoom est exprimé en pixels écran par unité monde**, et le zoom à la molette est **ancré sur le curseur** : le point du monde situé sous la souris reste fixe pendant l'agrandissement, comportement attendu de toute carte interactive. La caméra fournit aussi les conversions écran↔monde indispensables à la sélection des véhicules.

Un raffinement notable est le **lissage cinématique** : la caméra ne saute pas vers sa cible mais y glisse selon une interpolation exponentielle dont la demi-vie est réglable (200 ms par défaut). Lorsqu'on recentre la vue ou que l'on suit un véhicule, le mouvement reste doux et professionnel plutôt que brutal.

Shader — programmes GLSL

La classe `Shader` encapsule un programme GLSL (étage sommet plus étage fragment), gère sa compilation et son édition de liens, et **met en cache les emplacements d'uniformes** à leur première utilisation. Le projet compte cinq programmes de shaders : *basic*, *tile*, *graph*, *vehicle* et *link*.

5. Module cartographique — tuiles OpenStreetMap

Le module `map` affiche le fond de carte raster OpenStreetMap. C'est le sous-système le plus délicat car il combine **projection géographique**, **réseau asynchrone** et **gestion mémoire du cache GPU**.

MapProjection — projection Web Mercator

Toutes les coordonnées du monde sont exprimées en **mètres EPSG:3857** (Web Mercator), la projection standard des cartes en ligne. La classe `MapProjection` convertit latitude/longitude en mètres-monde et inversement, calcule l'emprise de chaque tuile, et choisit le **niveau de zoom OSM** dont les tuiles s'affichent à environ 256 pixels écran pour le zoom courant de la caméra. La latitude est bornée à ±85,05°, limite mathématique de la projection de Mercator.

TileLoader — téléchargement asynchrone

Le `TileLoader` gère un petit **pool de threads de travail** : chaque thread tire une demande de tuile dans une file, exécute la requête HTTP GET correspondante via libcurl, décode le PNG en RGBA, puis dépose le résultat dans une file de sortie. Le thread principal — seul propriétaire du contexte OpenGL — vide cette file et téléverse les pixels en textures. Le téléchargement réseau ne bloque donc jamais l'affichage. Un User-Agent valide est positionné, conformément à la politique d'usage des serveurs de tuiles OSM.

TileCache — cache LRU de textures

Le `TileCache` est un cache **LRU** (Least Recently Used) de tuiles déjà téléversées sur le GPU. Chaque tuile demandée est créée à l'état *Pending* et une demande de chargement est dispatchée ; une fois la capacité maximale dépassée, la tuile la moins récemment utilisée est évincée et sa texture GPU libérée. Les tuiles *Pending* ne sont jamais évincées, ce qui évite le phénomène de va-et-vient (*cache churn*).

TileRenderer — affichage

À chaque image, le `TileRenderer` détermine le niveau de zoom adapté, calcule le rectangle des tuiles visibles, interroge le cache (ce qui déclenche les chargements nécessaires) et dessine chaque tuile prête comme un quadrilatère texturé dans l'espace monde. Un plafond de tuiles demandées par image protège le serveur OSM d'une surcharge.

6. Graphe routier — import OSM et visualisation

OSMParser — lecture du fichier OpenStreetMap

L'`OSMParser` lit un fichier `.osm` (la sortie XML de l'API Overpass) à l'aide de pugixml. Il ne conserve que les éléments `<way>` dotés d'une étiquette routière `highway=*` exploitable pour la navigation ; tout le reste est ignoré. Les rues à double sens sont insérées comme une **paire d'arêtes dirigées**, les sens uniques et les ronds-points comme une seule arête. Les vitesses lues dans l'étiquette `maxspeed` (km/h ou mph) remplacent les valeurs par défaut associées au type de route.

RoadGraph — graphe routier compact

Le `RoadGraph` stocke nœuds et arêtes dans des `std::vector` plats indexés par entiers, choix qui favorise la **localité de cache** lors des parcours. La liste d'adjacence (arêtes sortantes par nœud) est construite paresseusement et invalidée à toute mutation. Chaque type de route possède une vitesse limite par défaut — autoroute 130 km/h, route primaire 80, voie résidentielle 30, etc. — et l'ordre de l'énumération sert aussi de priorité visuelle (les grands axes sont dessinés plus larges).

GraphRenderer — rendu des routes

Le `GraphRenderer` transforme le graphe en rubans de triangles colorés. La géométrie est **calculée une seule fois** après l'analyse — le réseau routier est statique — puis dessinée en un **unique appel de rendu** par image. Chaque arête devient deux triangles d'une largeur en mètres dépendant du type de route, et la couleur RGBA est pré-calculée par sommet.

> **Jeux de données fournis**
>
> Le dossier `data/` contient deux extraits OSM prêts à l'emploi : **mulhouse.osm** (11 Mo, idéal pour les essais rapides) et **la-rochelle.osm** (299 Mo, réseau étendu pour les tests de charge). N'importe quel extrait Overpass peut être chargé en argument de ligne de commande.

7. Simulation du trafic — véhicules et A*

Vehicle et VehiclePool — modèle de données

L'état canonique d'un véhicule n'est pas sa position, mais le couple **(arête courante, progression sur l'arête)** accompagné de son cap. La position en mètres-monde est une valeur recalculée à chaque pas et lue par les renderers. Chaque véhicule porte aussi un itinéraire (liste d'arêtes à parcourir), une distance au véhicule précédent, et de nombreux indicateurs d'événements : horodatage de la dernière diffusion V2V reçue, instant de fin d'immobilisation en cas d'accident, état de cession de passage à un véhicule d'urgence, compteurs de paquets émis et reçus, vitesse, accélération, kilométrage cumulé.

Le `VehiclePool` range tous les véhicules dans un `std::vector` contigu : la boucle de simulation traverse l'ensemble à chaque image, et la contiguïté mémoire maximise le débit. Les indices de véhicules servent de **poignées stables** — jamais réordonnées — pendant toute la simulation.

VehicleType — catégories de véhicules

Plusieurs catégories de véhicules coexistent (voitures, véhicules d'urgence, etc.), chacune décrite par une table de métadonnées `constexpr` : nom lisible, dimensions (longueur, largeur) en mètres, couleur et paramètres de comportement. Ces dimensions pilotent à la fois le rendu et la physique.

PathFinder — recherche de chemin A*

Le `PathFinder` implémente l'algorithme **A*** sur le graphe routier. Le coût minimisé est le **temps de trajet** en secondes : le coût d'une arête vaut sa longueur divisée par sa vitesse limite. L'heuristique est la distance euclidienne divisée par la vitesse la plus élevée du réseau, ce qui la rend **admissible** — elle ne surestime jamais le coût restant — et garantit donc que le chemin trouvé est réellement le plus rapide. Les tampons internes (scores, provenance, génération de visite) sont conservés et réutilisés d'un appel à l'autre : seule la file de priorité provoque une allocation.

SimulationEngine — moteur de pas

Le `SimulationEngine` possède le pool de véhicules et les fait tous avancer à chaque `step(dt)`. Lorsqu'un véhicule termine une arête, il passe à la suivante de son itinéraire ; si l'itinéraire est épuisé, il effectue une **marche aléatoire** aux intersections et une requête A* vers sa destination est mise en file d'attente. Point crucial pour la performance : le moteur **plafonne le nombre de recherches A* par image** (32 par défaut), ce qui lisse les pics de calcul processeur et maintient un temps d'image borné même avec des milliers de véhicules.

Le moteur calcule aussi, à chaque pas, la **distance au véhicule précédent** sur chaque arête (tri par arête puis balayage du plus proche voisin, en O(n log n) grâce à de petits seaux par arête). Cette distance sert à brider la vitesse et à empêcher les véhicules de se traverser.

8. Systèmes de trafic — feux, accidents, densité

Au-dessus du moteur de base, trois systèmes ajoutent du réalisme et de la dynamique au trafic.

TrafficLightSystem — feux de signalisation

Ce système contrôle des feux cycliques aux intersections. Seuls les nœuds comptant **au moins trois arêtes entrantes** — les véritables carrefours — sont gérés. Chaque intersection possède une liste ordonnée d'arêtes entrantes dont une seule est au vert à la fois, le vert tournant à intervalle régulier. Les véhicules interrogent `isGreen` pour décider s'ils doivent s'arrêter juste avant la fin de leur arête. Un bouton de l'interface permet de resynchroniser tous les feux.

AccidentSystem — génération d'accidents

L'`AccidentSystem` déclenche des accidents de façon probabiliste : à chaque tick, avec une probabilité proportionnelle à un taux réglable et au nombre de paires de véhicules en **suivi rapproché** (faible distance et vitesse élevée), une collision est provoquée. Les véhicules impliqués sont **figés en obstacles** pendant une durée donnée, forçant les suivants à faire la queue. Un accident peut émettre automatiquement une **diffusion V2V d'urgence**, et les épaves sont rendues par un marqueur rouge clignotant. L'interface offre un bouton « déclencher maintenant » pour la démonstration.

DensityHeatmap — carte de densité

La `DensityHeatmap` superpose une grille uniforme colorée (du bleu au rouge) indiquant le nombre de véhicules par cellule. L'accumulation est **lissée temporellement** pour que la carte ne scintille pas aux vitesses de simulation élevées, et seule une cellule non vide donne lieu à un quadrilatère dessiné. La taille de cellule par défaut est de 150 mètres.

9. Réseau V2V — liens radio et diffusions

Le module `v2v` est la raison d'être du projet : il modélise et visualise la communication entre véhicules.

V2VNetwork — calcul des liens

À chaque image, le `V2VNetwork` reconstruit l'ensemble des liaisons radio actives. L'algorithme interroge d'abord la grille spatiale pour obtenir les **paires candidates** proches — en temps linéaire moyen, sans le balayage en O(n²) que représenterait la comparaison de tous les couples. Pour chaque candidat, la portée effective est le **minimum des portées des deux véhicules** (un lien n'existe que si chacun peut entendre l'autre) ; si la distance est dans cette portée, la force du signal est calculée et le lien conservé.

RadioModel — modèle de signal

Le `RadioModel` est un modèle de liaison volontairement simple : la force décroît de façon **quadratique** entre 1,0 à distance nulle et 0 à la portée maximale. Ce n'est pas un modèle de propagation de qualité physique, mais il capture l'intuition « plus proche = plus fort » nécessaire à la visualisation, et reste aisément remplaçable. Une rampe de couleur rouge→jaune→vert traduit visuellement la force du lien, l'opacité s'estompant pour les liens faibles.

LinkRenderer — rendu des liens

Le `LinkRenderer` dessine toutes les liaisons actives comme des segments de droite, chaque extrémité portant la couleur issue de la force du signal. La géométrie est reconstruite côté processeur et téléversée dans un VBO dynamique — un seul appel de rendu par image.

Broadcast — diffusions multi-sauts

Le système de **diffusion** modélise la propagation d'un message émis par un véhicule (ou par un accident). Chaque diffusion est visualisée par un **anneau qui s'étend** à vitesse constante jusqu'à un rayon maximal. Quand le front d'onde dépasse un véhicule, celui-ci est marqué comme récepteur et brille brièvement. Le système gère la propagation **multi-sauts** : un véhicule touché peut relayer le message (génération 1, 2, …), chaque génération arborant une couleur distincte, avec un délai anti-rebond pour éviter qu'un message ne fasse indéfiniment l'aller-retour entre deux voisins.

10. Optimisation spatiale et performance

Atteindre 2000 véhicules à 60 images par seconde imposait d'éliminer tout algorithme quadratique du chemin critique. Deux techniques y répondent.

SpatialGrid — grille de hachage spatial

La `SpatialGrid` est une grille de hachage spatial 2D, **reconstruite à chaque image** puisque les véhicules bougent. La taille de cellule est choisie supérieure au plus grand rayon de requête, de sorte que toute recherche de proximité ne touche au plus que les 3×3 cellules voisines. Elle fournit deux requêtes essentielles : tous les identifiants dans un rayon donné, et toutes les paires de véhicules mutuellement proches — cette dernière étant la base du calcul du réseau V2V. Le coût passe ainsi de $O(n^2)$ à $O(n)$ en moyenne.

Rendu instancié des véhicules

Le `VehicleRenderer` utilise le **rendu instancié** d'OpenGL : un seul triangle unitaire est défini une fois, et les N véhicules sont dessinés en un appel grâce à un tampon d'instances contenant, par véhicule, sa position, son cap, sa couleur et sa taille. Chaque type de véhicule conserve ainsi son gabarit propre, et un facteur d'échelle global dépendant du zoom garde les véhicules visibles à toutes les distances. Tous les autres renderers (graphe, liens, diffusions) suivent la même philosophie : **un appel de rendu par couche**.

Profiler — mesure intégrée

Le `Profiler` est un profileur de sections à portée (RAII) : une macro `V2V_PROF` chronomètre automatiquement le bloc englobant. Le projet est instrumenté avec **près de 30 sections nommées** couvrant chaque phase de simulation et de rendu (pas de simulation, grille spatiale, mise à jour V2V, rendu des tuiles, du graphe, des véhicules, des liens, de l'interface, etc.). Les temps sont moyennés sur une fenêtre glissante d'une seconde et affichés dans le panneau de performance, ce qui rend tout goulet d'étranglement immédiatement visible.

11. Interface utilisateur ImGui

ImGuiLayer — intégration

La `ImGuiLayer` est une enveloppe RAII autour de Dear ImGui et de ses backends GLFW et OpenGL 3. Elle s'initialise après la création du contexte graphique et libère proprement l'ensemble à la destruction. ImGui fonctionne en **mode immédiat** : l'interface est entièrement redéclarée à chaque image, ce qui élimine toute désynchronisation entre l'état affiché et l'état réel de la simulation.

Panels — panneaux de contrôle

Le fichier `Panels.cpp` — le plus volumineux du projet — dessine tous les panneaux : contrôles généraux, gestion des couches d'affichage, réglages de la simulation, état du réseau V2V, mesures de performance, et un inspecteur de véhicule sélectionné. Les pointeurs vers les sous-systèmes y sont **non possédants et peuvent être nuls** : sans graphe chargé, les panneaux de simulation et de réseau se désactivent proprement plutôt que de planter.

L'inspection est **bidirectionnelle** : cliquer un véhicule le sélectionne et ouvre son panneau (type, vitesse, accélération, paquets émis/reçus, itinéraire), et le bouton « désélectionner » referme la boucle. Un panneau permet aussi d'émettre une diffusion V2V depuis le véhicule sélectionné, et l'application a évolué vers un véritable **centre de contrôle** regroupant les boutons de bascule des couches et un radar intégré.

Commandes clavier et souris

Commande	Action
Clic droit + glisser	Déplacer la vue (pan)
Molette	Zoomer / dézoomer, ancré sur le curseur
R	Recentrer la caméra sur le graphe chargé
G / M / V / L	Afficher graphe / carte / véhicules / liens V2V
Espace	Mettre en pause ou reprendre la simulation
+ / - / 0	Échelle de temps ×2, ÷2, ou retour à ×1
F1	Activer la surimpression de diagnostic
Échap	Quitter l'application

Les entrées clavier et souris sont automatiquement **neutralisées** lorsque le curseur survole un panneau, afin que cliquer dans l'interface ne déplace pas la carte.

12. Météo et effets d'ambiance

Le système météo apporte une couche d'immersion qui n'est pas que cosmétique : il agit aussi sur la **physique** de la simulation.

Weather — état météo global

La structure `Weather` centralise un état global — ciel clair, pluie ou brouillard — qui pilote deux effets mesurables. D'une part, un **multiplicateur de portée V2V** : la pluie dégrade légèrement la portée des transmissions, le brouillard plus fortement (diffusion et absorption par les gouttelettes). D'autre part, un **multiplicateur de vitesse** des véhicules : chaussée humide et mauvaise visibilité incitent à ralentir. Un même réglage influence donc simultanément le rendu et le comportement.

RainRenderer — pluie procédurale

Le `RainRenderer` est un système de particules procédural : des gouttes vivent dans un rectangle virtuel un peu plus grand que la vue, tombent à vitesse fixe, et se réenroulent en haut lorsqu'elles sortent du cadre. L'intensité (de 0 à 1) contrôle linéairement le nombre de gouttes effectivement dessinées, ce qui borne le coût de rendu. Le brouillard est rendu séparément comme un voile translucide superposé à la scène.

> **Journalisation**
>
> Le fichier `utils/Log.h` fournit des macros de journalisation minimalistes (`V2V_LOG_INFO`, `WARN`, `ERROR`) construites sur `__VA_OPT__`, une fonctionnalité standard du C++20 qui insère la virgule des arguments variadiques sans générer d'avertissement — contrairement à l'ancienne astuce GNU.

13. Méthode de développement par étapes

Le projet a été construit selon un **plan séquentiel**, chaque étape devant aboutir à une démonstration exécutable avant de passer à la suivante. Cette discipline a garanti qu'à tout moment le code compilait et produisait un résultat observable, plutôt que d'accumuler des morceaux non intégrés.

Étape	Livrable
1 — Initialisation	CMake, arborescence, dépendances, fenêtre noire
2 — Moteur graphique	Fenêtre, renderer, caméra pan/zoom, boucle delta-temps
3 — Cartographie	Projection Mercator, cache LRU, chargement de tuiles
4 — Graphe routier	Analyseur OSM, graphe, visualisation des routes
5 — Simulation	Véhicules, moteur de pas, recherche de chemin A*
6 — Optimisation	Grille spatiale, rendu instancié, cible 2000 véhicules
7 — Réseau V2V	Réseau de liens, modèle radio, rendu des liaisons
8 — Interface	Panneaux ImGui de contrôle, de mesure et de diagnostic

Le projet a ensuite **dépassé son plan initial de huit étapes**. Les itérations suivantes ont ajouté les feux de signalisation, le système d'accidents, la carte de densité, la météo et la pluie, les diffusions multi-sauts, le profileur intégré, les catégories de véhicules et l'inspecteur, jusqu'à un centre de contrôle complet doté d'un radar. Chaque ajout a respecté la même règle : un livrable démontrable, intégré et compilable.

> **Principe de conception retenu**
> Séparer rigoureusement l'état (la simulation, sans aucune dépendance OpenGL) du rendu (qui lit cet état). C'est ce qui a permis d'ajouter sept systèmes supplémentaires après le plan initial sans jamais déstabiliser le noyau de simulation.

14. Statistiques du projet

Quelques chiffres caractérisant l'ampleur et la structure du code à la date de ce rapport.

Indicateur	Valeur
Lignes de code C++ (.cpp + .h)	8 318
Fichiers d'implémentation (.cpp)	30
Fichiers d'en-tête (.h)	34
Programmes de shaders GLSL	5 (basic, tile, graph, vehicle, link)
Modules fonctionnels	7 (graphics, map, graph, simulation, v2v, ui, utils)
Sections instrumentées par le profileur	~30
Cible de performance	2000+ véhicules à 60 images/seconde
Norme du langage	C++20
API graphique	OpenGL 3.3 Core Profile

Répartition du code par module

Module	Lignes	Part
ui	1 732	21 %
simulation	1 618	19 %
map	926	11 %
graphics	857	10 %
utils	848	10 %
v2v	718	9 %
graph	713	9 %
main + divers	~906	11 %

L'interface et la simulation concentrent à elles seules 40 % du code, ce qui reflète la richesse fonctionnelle ajoutée après le plan initial : panneaux d'inspection, centre de contrôle, et l'ensemble feux / accidents / densité côté simulation.

15. Vérification, état du build et pistes d'évolution

Vérification statique effectuée

Une vérification du code a été menée. Les points contrôlés et leurs résultats :

- **Cohérence du build** : les 31 fichiers source listés dans `CMakeLists.txt` existent tous sur le disque.
- **Inclusions** : chaque fichier `.cpp` inclut bien son propre en-tête ; aucun en-tête manquant détecté.
- **Propreté** : aucune marque `TODO` ni `FIXME` laissée dans le code source.
- **Correction appliquée** : `main.cpp` contenait deux directives `#include` dupliquées (`AccidentSystem.h` et `DensityHeatmap.h`) ; les doublons ont été supprimés.
- **Point relevé** : les fichiers `utils/HeatmapRenderer.cpp/.h` ne sont ni compilés ni référencés — code orphelin remplacé par `simulation/DensityHeatmap`. Ils peuvent être supprimés sans risque.

État de la compilation

La compilation complète n'a pas pu être exécutée dans l'environnement d'analyse, qui ne dispose ni des droits d'administration ni des bibliothèques de développement système requises (`libcurl-dev`, en-têtes OpenGL). Le projet doit être compilé sur le poste de développement habituel, où ces dépendances sont présentes, avec les commandes du chapitre 2. La vérification statique ci-dessus n'a révélé aucune anomalie bloquante.

Pistes d'évolution

- Remplacer le `RadioModel` quadratique par un modèle de propagation log-distance ou de type Friis, sans toucher au reste du module V2V (l'interface a été pensée pour ce remplacement).
- Supprimer le code orphelin `HeatmapRenderer` pour alléger l'arborescence.
- Épaissir les liens V2V en passant des segments `GL_LINES` à la technique des rubans de triangles déjà employée par le `GraphRenderer` (la largeur de ligne supérieure à 1 pixel n'est pas honorée sur macOS).
- Substituer au serveur de tuiles OSM public un serveur auto-hébergé ou un fournisseur commercial pour un usage soutenu.

Rapport généré automatiquement — Projet V2V, Simulateur de communication Vehicle-to-Vehicle — Mai 2026.